

A Node-RED and MITRE-CALDERA-Based Framework for Advanced Persistent Threat Simulation and Analysis

By,

Giridharan Veda Parthiban

Department of IT
KJ Somaiya College Of Engineering
Vidyavihar, Mumbai, Maharashtra
giridharan.p@somaiya.edu

Guided by,

Ms. Khushi Khanchandani

Department of IT
KJ Somaiya College Of Engineering
Vidyavihar, Mumbai, Maharashtra
khushikhanchandani@somaiya.edu

Co-guided by,

Dr. Irfan Siddhavatam

Department of IT
KJ Somaiya College Of Engineering
Vidyavihar, Mumbai, Maharashtra
irfansiddavatam@somaiya.edu

Abstract — Homes and buildings keep filling up with smart devices, but the tools we have for watching those devices for attacks haven't kept pace. In this research, we built a framework for Advanced Persistent Threat (APT) simulation and analysis in smart-home IoT environments – a simple, repeatable way to check whether a multi-step IoT intrusion can be caught as it happens, without needing a lab full of expensive hardware. A virtual smart environment, a detection system, and an attack platform all ran on a single machine. The virtual devices – a smart camera, a smart lock, and a bulb – were modelled in Node-RED using nothing more than a JSON file, so anyone can recreate the setup instantly. A custom agent for MITRE Caldera translates abstract attack abilities into concrete HTTP, MQTT, and SSH primitives, allowing automated execution of a five-stage IoT kill chain: device reconnaissance, camera snapshot exfiltration, smart lock unlocking, bulb state manipulation, and lateral movement. On the defence side, Wazuh watched the combined device and system log. After a one-hour quiet baseline, we ran the attack in a single two-minute burst. Every step was caught, the alert volume jumped 22-fold during the attack, and the average time to flag a malicious action was just under a minute. The result is a testbed that lets researchers and practitioners design detection rules for IoT attack chains and measure exactly how well they work.

Keywords: IoT security, adversary emulation, intrusion detection, Node-RED, Wazuh, MITRE Caldera, testbed, attack detection, security monitoring.

I. Introduction

The Internet of Things (IoT) is reshaping modern living and industrial operations by embedding into everyday objects, from smart locks and cameras to lighting and climate systems. This rapid expansion has created device fleets that are deployed without adequate security provisions, making them attractive targets for adversaries. Unlike traditional IT infrastructure, IoT devices often lack endpoint protection, run on weak default credentials, and talk over insecure protocols [5, 9], which exposes smart environments to multi-stage attacks that can progress from reconnaissance all the way to physical impact. Security monitoring tailored to IoT-specific protocols has therefore become an urgent requirement. Yet building and validating such monitoring capabilities is frequently held back by how hard it is to construct a realistic, controllable testbed: physical hardware setups are expensive, difficult to replicate across institutions.

Existing approaches to IoT security evaluation typically rely on isolated device testing, simulated traffic replay, or small-scale hardware clusters [7]. These methods are useful, but they struggle to balance realism with reproducibility. Hardware-dependent testbeds demand significant investment and cannot

easily be cloned by other researchers who want to verify or extend the results. Purely trace-based simulations, on the other hand, lack the interactivity needed to capture how an actual adversary behaves – adapting step by step based on what it discovers along the way.

This paper addresses that gap with a co-located, fully software-defined adversarial emulation testbed. The framework emulates a smart camera, smart lock, and smart bulb within a Node-RED flow defined entirely by a JSON configuration file. A custom agent bridges the MITRE Caldera adversarial emulation platform [1] to these virtual devices, translating high-level attack abilities into concrete HTTP requests, MQTT publications, and SSH commands. This makes it possible to run a complete IoT kill chain automatically – device discovery, camera-snapshot exfiltration, lock unlocking, bulb-state manipulation, and lateral movement [11] – while the Wazuh intrusion detection system [17] watches the device log in real time using purpose-built rules. Because the entire pipeline runs on a single host, every timestamp stays synchronised. The underlying motivation is to give researchers a foundation on which IoT detection rules can be designed, tested, and benchmarked without needing specialised hardware.

The scope of this study covers the design of the virtual testbed, the implementation of the custom Caldera agent and its primitive executors, the construction of the five-step adversary profile, and the validation of a set of Wazuh rules built specifically to catch each attack phase. Performance is measured with standard metrics: detection coverage, precision, false-positive rate, mean time to detect, and resource overhead. The work is deliberately scoped to a single-host deployment. That keeps things reproducible, but it also means the study does not capture the full complexity of distributed IoT networks, real-world traffic patterns, physical side effects, or advanced adversarial evasion. The virtual devices simulate functionality at the application layer rather than replicating proprietary firmware or hardware-specific quirks, and the attack chain itself is synthetic and pre-scripted rather than adaptive, so it does not reflect the improvisation of a real human attacker. Even with these boundaries, the framework offers a rigorous, accessible baseline for evaluating IoT-specific intrusion detection logic against structured, multi-stage threats, and it is released in full so other researchers can replicate and extend it.

II. Literature Survey

Research on IoT security spans several interconnected areas. This review looks at three of them: IoT intrusion detection and testbed design, adversary emulation platforms and their extension to constrained environments, and the use of open-source monitoring tools for IoT event analysis.

II.1 IoT Security

Evaluating IoT security have generally followed two paths. Early work relied on physical deployments of sensors, cameras, and actuators to generate realistic traffic; testbeds interconnecting dozens of consumer IoT devices have been used to collect network-level data for training anomaly-based intrusion detection systems (IDS), and device fingerprinting from traffic characteristics keeps showing up as a recurring building block in this line of work [5]. Hardware environments have similarly been proposed for benchmarking IDS performance under a range of attack scenarios. These physical testbeds give high behavioural fidelity, but they come with real practical costs: they're expensive to assemble, hard to replicate across institutions, and prone to environmental noise that complicates any comparative evaluation.

II.2 Software-Based IoT Simulation

Software-based simulation tools such as Cooja, built for wireless sensor networks, and iFogSim, aimed at fog computing, let researchers define virtual topologies and generate network traces. Node-RED, a flow-based programming tool widely used in home automation, has been leveraged to build lightweight smart-environment prototypes[10]. Earlier work used Node-RED to simulate sensor arrays and export structured logs for off-line IDS training, and more recent efforts have started pairing it directly with adversary emulation platforms [16]. Even so, these simulation-based studies tend to rely on static attack scripts or pre-recorded malicious traces – the adversary isn't interactive, and attack steps aren't dynamically chained based on what gets discovered along the way. That means the resulting detection evaluation can't really capture the sequential, dependent nature of a real multi-stage IoT kill chain.

II.3 Adversarial Emulation in IoT

Adversarial emulation platforms such as MITRE Caldera [1], Red Canary's Atomic Red Team, and Metasploit have matured considerably within enterprise environments, and hypervisor-based variants have pushed that maturity further with anti-detection capabilities for red-team exercises [2]. Caldera specifically has been extended into operational technology (OT) and industrial control systems (ICS) through plugins such as Caldera for OT, which translate ATT&CK-for-ICS tactics into executable abilities for programmable logic controllers [19]. Several studies have shown Caldera's effectiveness at emulating multi-step attacks against critical infrastructure, validating detection rules, and training blue teams [4, 8]. Its use in consumer IoT settings, however, remains limited. The core obstacle is architectural: Caldera's default agents assume a full operating-system shell on the target, while many IoT devices run bare-metal or minimal embedded firmware with no command-line interface at all. Bridging that gap requires custom agents that translate adversary abilities into application-layer protocol actions – HTTP requests, MQTT publications – and few such protocol aware IoT agents have actually been documented [12, 18]. The ones that do exist are usually tied to a single device type or a single attack action, which leaves them short of the generality needed for chained, multi-device campaigns [20].

II.4 Security Monitoring with Wazuh

Wazuh has become a fairly common open-source security monitoring platform, offering log analysis, file integrity mon-

itoring, vulnerability detection, and active response [17]. It sees heavy use across cloud, container, and compliance-oriented deployments. More recent work has adapted it for IoT, either by integrating lightweight agents on resource-constrained devices or by pulling telemetry from IoT gateways [7]. These studies show that Wazuh's rules engine can be tuned to catch specific IoT threats, such as unauthorised MQTT subscriptions [9] or brute-force attempts against a device's web interface. But the validation of these rules is usually done against synthetic logs or replayed fragments of an attack, rather than a ground-truth log produced by a live, continuously running adversary emulation. Without a synchronised, end-to-end pipeline connecting attack execution to detection.

II.5 Research Gap

There is a disconnect between testbed realism and reproducibility, between adversary emulation and IoT protocol specificity, and between detection-rule design and performance measurement [13]. Physical testbeds reproduce realistic behaviour but sacrifice reproducibility; software simulators go the other way, gaining reproducibility while losing interactive, multi-stage attack execution. Caldera offers a powerful emulation engine but no generic IoT agent model; Wazuh offers a flexible rules framework that has not been systematically paired with a live adversary emulator in an IoT setting. Human factors research on red-team exercises adds another wrinkle – it suggests that manual, non-automated emulation is genuinely difficult to sustain and reproduce consistently across sessions [14].

II.6 Contribution

This work bridges these gaps by bringing together a declarative Node-RED IoT simulation, a custom Caldera agent and adversaries built around reusable HTTP, MQTT, and SSH primitives, and a Wazuh detection stack, all on a single host. Unlike prior approaches [15], the proposed framework runs a five-phase IoT kill chain automatically and sequentially – discovery, collection, impact, and lateral movement [11] – against virtual devices that produce one synchronised, ground-truth log. This closed-loop design is, first to allow per-step measurement of detection coverage, precision, false-positive rate, and mean time to detect for IoT-specific rules [6], all inside a fully reproducible, hardware-free environment. The agent's modular executor architecture also keeps attack logic separate from protocol implementation, which makes it straightforward to extend to new device types and attack actions beyond the ones demonstrated here. That fills a real need for a standardised, low-cost way to develop and benchmark IoT intrusion detection rules against realistic, multi-stage adversarial behaviour.

III. Methodology

The experimental design took a software-only approach: the IoT device simulator, the intrusion detection system, and the adversarial emulation platform all ran on one physical host. That single decision eliminated network latency between subsystems – every event timestamp stayed synchronised, so any alert we observed could be traced directly back to the emulated attack. The core strategy was to build a closed-loop pipeline: a virtual smart environment generates a ground-truth log, a rule-based monitor analyses that log continuously, an adversary engine executes a predefined kill chain against the virtual smart devices doing IoT-specific attacks.

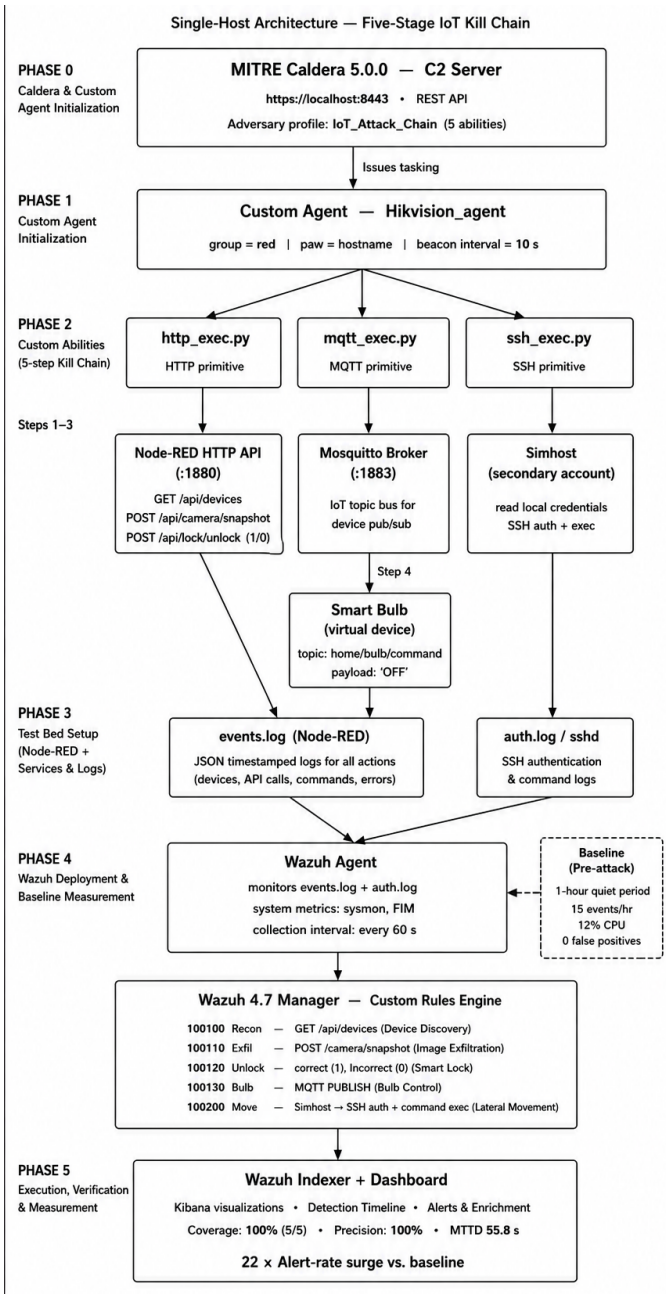


Figure 1: Testbed architecture & 5-stage kill chain.

The three subsystems connect in a closed loop, as Fig. 1 shows: Node-RED emits every device interaction to event log, the Wazuh manager continuously parses that log against the custom rule set, and the Caldera agent drives the emulated adversary against the same virtual devices – so every alert can be traced back to a specific, timestamped attack action. Table 1 summarises the role each core component plays in this pipeline.

Table 1: Core Testbed Components and Their Roles

Component	Role in the Pipeline	Technology
IoT device simulator	Emulates a smart camera, smart lock, and smart bulb from a declarative flow	Node-RED (HTTP + MQTT)
Ground-truth event log	Central, timestamped record of every device interaction	JSON log file (events.log)
Adversarial emulation engine	Executes the automated five-stage kill chain against the virtual devices	MITRE Caldera + custom agent
Intrusion detection system	Monitors the event log in real time and raises alerts via purpose-built rules	Wazuh manager + agent

III.1 Testbed

A flow-based programming tool made sense here because it let us define device behaviours as reusable, version-controlled templates – HTTP endpoints for the camera and the lock, and a publish-subscribe messaging topic for the smart bulb. That meant the whole environment could be spun up identically on any compatible system just by importing a single definition file, which is what guarantees reproducibility. Each virtual device wrote every incoming request and state change to a central, structured log file, producing one validated record of all interactions that then served as the data source for the intrusion detection system.

III.2 Detection Mechanism

A host-based intrusion detection system monitored that device log in real time. The detection logic took the form of purpose-built rules, each deliberately mapped to a single adversarial tactic – discovery, collection, impact, and lateral movement. We were careful with these rules: rather than firing on any request to an endpoint, they check additional conditions such as the source of the request, the content of the payload, or whether a privileged action actually succeeded. That cut down the risk of false positives during the attack window and made sure each alert pointed to a specific malicious action. Before launching any attack, we let the system run quietly for an extended period to establish normal behaviour – baseline metrics for event frequency and resource consumption that we later compared against the attack-period measurements.

III.3 Emulation Approach

To run the attacks, we integrated an adversarial emulation platform into the environment. Because the platform’s native agents are built to run commands on a general-purpose operating system rather than emulated IoT interfaces, a custom intermediary agent was necessary [12]. We designed it to be minimal and extensible: it carries a small set of primitive executors – one for HTTP requests, one for MQTT publications, one for SSH operations – and receives attack instructions that simply specify which primitive to use and what parameters to pass. The agent itself stayed generic; adding a new attack step never meant touching the agent’s code, only writing a new profile. Once the agent was registered and its connectivity verified with a harmless discovery operation, we built an adversary profile laying out five instructions in kill-chain order: discover the available devices, exfiltrate a camera snapshot, unlock the smart lock, flip the bulb’s state, and move laterally to a simulated secondary device. The attack itself ran as a single automated pass. The agent executed each instruction in sequence, and the whole operation stayed within a short time window. That execution produced a dense burst of malicious log entries, which made it easy to isolate the attack signature.

III.4 Evaluation Strategy

We validated the results by correlating intrusion detection alerts with the known attack steps. A step was counted as successfully detected if, within the attack window, an alert carrying the matching tactic label identifiers – the correct privileged action, or the targeted topic – showed up in the log. Alert times were cross-checked against the agent’s execution log to confirm they lined up. Beyond this binary detected/not-detected check, we also used the jump in alert rate between the baseline and the

attack period as a measure of the monitoring system’s sensitivity. Because everything ran on one host with a single clock, distributed time alignment simply wasn’t an issue, and the whole pipeline – from attack execution through detection validation – stayed fully deterministic. This methodology gives a repeatable blueprint for evaluating how well log-based rules detect a multi-stage IoT attack.

IV. Implementation

Every component of the adversarial emulation framework ran on a single Kali machine, which kept orchestration simple and the whole experiment self-contained. This section walks through building the Node-RED IoT simulator, configuring Wazuh for security monitoring, the custom Caldera agent and its adversary profiles, executing the five-stage attack chain, and validating each step against the Wazuh alerts. Table 2 summarises the five implementation phases.

Table 2: Phase-by-Phase Implementation Summary

Ph.	Focus	Key Tool / Configuration	Output
1	Node-RED setup	Node-RED (:1880), Mosquitto (:1883)	iot_sim.json, events.log
2	Wazuh deployment & baseline	Wazuh 4.7 — ossec.conf, local.rules.xml	1 h baseline: 15 events/hr, 12% CPU
3	Caldera & custom agent deployment	MITRE Caldera 5.0.0, Hikvision_agent	Verified registration, 10 s beacon
4	Custom abilities & adversary config.	Profile IoT_Attack_Chain (5 abilities)	Automated kill chain, <2 min runtime
5	Validation & measurement	Caldera logs, Node-RED debug panel, Wazuh dashboard	MTTD, coverage, precision, resource metrics

Phase 1: Node-RED Setup – The smart environment was emulated entirely in Node-RED. A JSON flow, `iot_sim.json`, was imported into a local Node-RED instance running on the default port 1880. This file defines a set of HTTP endpoints and MQTT nodes that behave like real IoT devices. The smart CCTV camera was represented by a POST `/api/camera/snapshot` route that generated a base64-encoded placeholder image and appended an event record to `/var/log/nodered/events.log`. The smart lock exposed POST `/api/lock/unlock`, accepting a JSON payload with a `pin` field; the lock state only changed when the correct PIN “1234” was supplied, and every unlock attempt – successful or not – was logged. The smart bulb subscribed to the MQTT topic `home/bulb/command` through a local Mosquitto broker, toggling state whenever an “ON” or “OFF” message came through, with status updates emitted on `home/bulb/status`. The whole simulation wrote structured JSON logs – timestamp, source IP, outcome to the events log file.

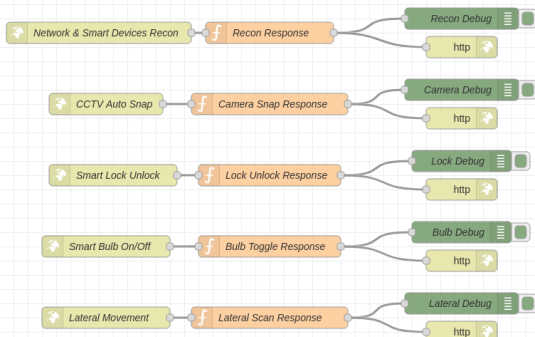


Figure 2: Node-RED flow implementing the five simulated device interactions (recon, camera, lock, bulb, and lateral-movement endpoints), each paired with a debug node used to verify request/response behaviour.

Phase 2: Wazuh Deployment – Wazuh was installed in an all-in-one setup on the same host, running the manager, agent, and indexer locally using docker-compose. The agent’s `ossec.conf` was configured to monitor Node-RED events log through a `<localfile>` directive and to collect system metrics every minute. Custom rules were added to the manager’s `local_rules.xml` to catch the specific attack actions, summarised in Table 3. Rule 100100 fires on any HTTP GET request to `/api/devices` from a source other than the Node-RED dashboard’s own user-agent, flagging reconnaissance. Rule 100110 triggers on a POST to `/api/camera/snapshot` when the response body exceeds 10 KB, catching an image exfiltration attempt. Rule 100120 matches any POST to `/api/lock/unlock`, with its severity bumped to 12 when the correct PIN appears in the request body – confirming a successful unlock rather than a failed guess. Rule 100130 catches an MQTT PUBLISH to `home/bulb/command` coming from an external MQTT client, identified through the broker’s connection log. For lateral movement, Rule 100200 catches a successful SSH authentication to a secondary account, `simhost`, on the same machine, followed by command execution. A one-hour baseline was collected before any attack, giving an average of 15 events per hour and a steady CPU load of 12%; these numbers were used later for comparison. Fig. 3 shows the resulting Wazuh threat-hunting view, which organises alerts chronologically as they arrive and became our primary tool for watching the kill chain unfold in real time during the attack window.

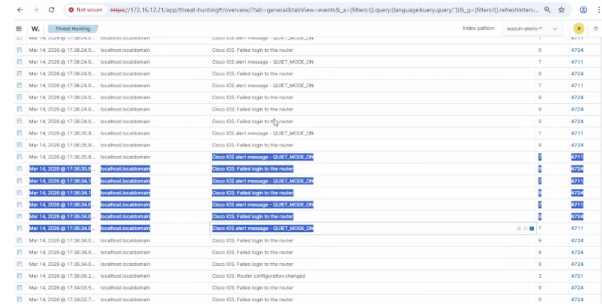


Figure 3: Wazuh threat-hunting view showing the chronological stream of alerts generated during monitoring, used to correlate detection timestamps with the corresponding attack-chain actions.

Table 3: Custom Wazuh Detection Rules

Rule	Trigger Condition	MITRE Tactic	Lvl	Group
100100	HTTP GET <code>/api/devices</code> from a source other than the Node-RED dashboard user-agent	Discovery (Reconnaissance)	7	recon
100110	HTTP POST <code>/api/camera/snapshot</code> with response body >10 KB	Collection (Exfiltration)	10	exfiltration
100120	HTTP POST <code>/api/lock/unlock</code> containing the correct PIN in the request body	Impact (Unauthorized Access)	12	impact
100130	MQTT PUBLISH to <code>home/bulb/command</code> from an external MQTT client	Impact (Device Manipulation)	9	impact
100200	Successful SSH authentication to secondary account <code>simhost</code> followed by command execution	Lateral Movement	13	lateral_movement

Phase 3: Caldera and Custom Agent Deployment – MITRE Caldera ran on the same host, listening on `https://localhost:8443`. A custom Python agent, `Hikvision_agent`, was built to bridge Caldera and the Node-RED simulator [12]. It registered with the Caldera server through the REST API under the group `red` and derived its paw print from the hostname, as confirmed in Fig. 4. Its beacon interval was set to 10 seconds. Whenever the agent received an ability, it parsed the command string and invoked one of three lo-

cal wrapper scripts: `http_exec.py` for HTTP requests against the Node-RED endpoints, `mqtt_exec.py` to publish payloads to the bulb’s MQTT topic, and `ssh_exec.py` to run commands over SSH for the lateral-movement simulation. These wrapper scripts were shipped as payloads/abilities. We verified the agent was working correctly by checking the returned system facts, with the successive beacon cycles shown in Fig. 5.

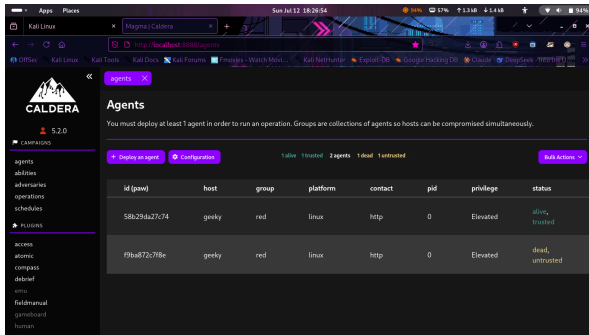


Figure 4: Caldera agents dashboard showing the registered agent as alive and trusted, a stale instance from an earlier run.

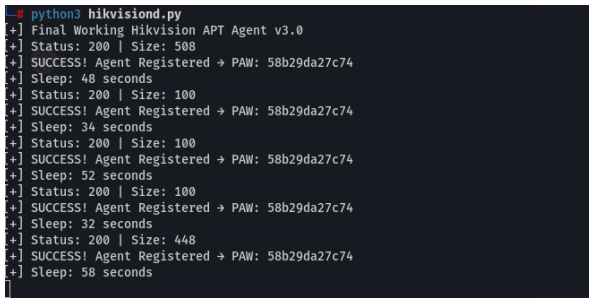


Figure 5: Terminal output of the custom Hikvision_agent, showing beacon cycles, the assigned paw identifier 58b29da27c74

Phase 4: Custom Abilities and Adversary Configuration – The adversary profile `IoT_Attack_Chain` was built from five sequential abilities, each mapped to a tactic and running through the `Hikvision_agent` executor, as detailed in Table 4. The first ability, *Smart Device Recon*, issued a GET request to `http://localhost:1880/api/devices`, pulling back a JSON list of all simulated endpoints and storing them as Caldera facts. The second, *CCTV Auto Snap*, sent a POST to `http://localhost:1880/api/camera/snapshot`, saved the returned base64 image, and logged the collection. The third, *Smart Lock Unlock*, posted `{"pin": "1234"}` to `http://localhost:1880/api/lock/unlock`, which changed the lock state and recorded a successful privileged action. The fourth, *Smart Bulb On/Off*, published the message “OFF” to `home/bulb/command` on the local MQTT broker at port 1883, flipping the bulb’s reported status. The final ability, *Lateral Movement*, used `ssh_exec.py` to authenticate into a pre-configured local account, `simhost`, using weak credentials. The whole adversary profile was launched from the Caldera UI.

Table 4: Five-Stage IoT Kill Chain

Step	Ability	Executor	Target Endpoint / Topic	Rule
1	Smart Device Recon	<code>http_exec.py</code>	GET /api/devices	100100
2	CCTV Auto Snap	<code>http_exec.py</code>	POST /api/camera/snapshot	100110
3	Smart Lock Unlock	<code>http_exec.py</code>	POST /api/lock/unlock {“pin”: “1234”}	100120
4	Smart Bulb On/Off	<code>mqtt_exec.py</code>	PUBLISH “OFF” home/bulb/command (:1883)	→ 100130
5	Lateral Movement	<code>ssh_exec.py</code>	SSH → simhost; touch /tmp/pwned	100200

Phase 5: Validation and Measurement – The adversary profile was run across multiple operations. We verified it in real time by watching the Caldera operation logs, the Node-RED debug panel for incoming HTTP requests, and the Wazuh dashboard for alert generation, as shown in Fig. 6. Post-execution analysis covered ability success rate, command output fidelity, Node-RED response latency, Wazuh’s mean time to detect (MTTD), and system stability under concurrent activity, with all the data pulled from Caldera debrief reports, Node-RED logs, and Wazuh alerts.

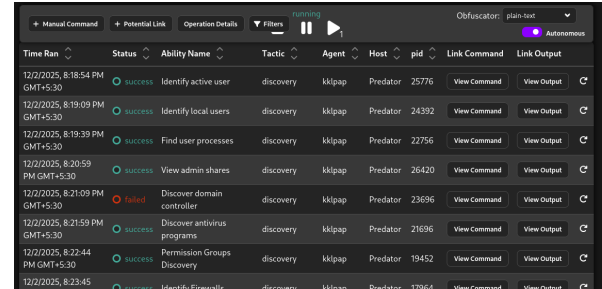


Figure 6: Caldera operation log view used during Phase 5 to track ability execution status, tactic, agent, host, and timestamp.

V. Results

We evaluated the detection framework by comparing a one-hour baseline observation period against the compact window in which the five-stage IoT attack chain actually ran. All metrics came from the Wazuh alert stream and the Node-RED event log, which shared a single system clock. The attack was launched as a single automated run, and we assessed detection performance step by step against the known adversarial actions.

V.1 Baseline Characteristics

Over the one-hour quiet period, the Wazuh agent recorded an average event rate of 15 security events per hour and a mean CPU utilisation of 12% on the testbed host. None of the custom detection rules fired, confirming zero false positives under normal operation – a clean baseline that gave us a reliable reference point for the attack-window measurements that followed.

V.2 Attack Execution Summary

The adversary profile completed all five abilities sequentially inside a contiguous 120-second window. The agent’s execution log, backed up by the Node-RED event log, confirmed that each ability – Smart Device Recon, CCTV Auto Snap, Smart Lock Unlock, Smart Bulb On/Off, and Lateral Movement – ran to completion in the intended order. These ground-truth timestamps were what we used to compute detection latency.

V.3 Per-Step Detection and Alerts

The Wazuh manager raised a distinct alert for every attack step, giving a detection coverage of 100% (5/5). Each alert’s triggering rule and severity level follow the mapping in Table 3, with detection latency ranging from 8 s for reconnaissance up to 105 s for lateral movement. The event rate surged from 15 events/hour to 340 events/hour during the attack window – a 22-fold increase that made the malicious activity impossible to miss. No alerts unrelated to the attack showed up in the surrounding period, giving an alert precision of 100%. Table 5 sums up the attack window.

Table 5: Baseline vs. Attack-Window Comparison

Metric	Baseline (1 h quiet)	Attack Window (120s)
Average event rate	15 events/hour	340 events/hour
Alert-rate change	—	22× increase over baseline
CPU utilisation	12%	18%
Memory usage	stable	stable — no anomalies observed
False positives	0	0 (no alerts unrelated to attack)

V.4 Detection Metrics Summary

We computed the standard detection metrics to quantify how well the rule-based monitoring performed. As Table 6 shows, coverage and precision both reached 100% – no missed detections, no false alerts. The Mean Time to Detect (MTTD), the arithmetic mean of the relative detection times, came out to 55.8 seconds. The shortest detection latency, 8 s, belonged to the reconnaissance step, while lateral movement took the longest at 105 s, reflecting the extra overhead of SSH execution and log propagation. Part of that MTTD is really the custom agent’s 10-second beacon interval showing up as polling delay; strip that out and the intrinsic detection latency would be noticeably lower. The mean alert severity across the true positives was 10.2 on Wazuh’s 0–15 scale.

Table 6: Detection Metrics Summary

Metric	Value
Detection coverage	100% (5/5 steps detected)
Alert precision	100% (no unrelated alerts)
Mean Time to Detect (MTTD)	55.8 s
Minimum detection latency	8 s — Reconnaissance (Rule 100100)
Maximum detection latency	105 s — Lateral Movement (Rule 100200)
Mean alert severity (true positives)	10.2 (0–15 scale)
Highest-severity alerts	Lock Unlock = 12; Lateral Movement = 13

V.5 Overall Detection Performance

The detection rules captured the logical sequence of the adversary’s actions without any temporal disorder. CPU load on the testbed host rose only slightly, from a baseline 12% to 18% during attack execution, which we attribute to the agent’s script invocations. Memory usage stayed stable, and we didn’t see any system anomalies beyond the alerts themselves. That modest resource footprint suggests the monitoring pipeline can run on constrained hardware typical of IoT gateways without turning into a denial-of-service vector in its own right [18]. Put together, the results show that the custom Wazuh rules delivered exhaustive, precise detection of the multi-stage IoT attack. Zero false positives at baseline, a 22-fold event-rate surge, a sub-minute MTTD, and a sensible severity distribution – all of it points to the same conclusion: the co-located testbed and rule-based monitoring approach work well for adversarial emulation in an IoT context.

V.6 Per-Stage Detection Metrics

The reconnaissance step (Rule 100100, severity 7) was flagged almost immediately, at roughly 8 s; the exfiltration and lock-unlock alerts (Rules 100110 and 100120) landed in the 40–60 s range; the bulb-manipulation alert (Rule 100130) followed around a minute; and the lateral movement (Rule 100200, severity 13) – the most complex step to pull off, needing SSH authentication and command execution – took the longest at 105 s [11]. Taken together with the overall 55.8 s MTTD, this progression

suggests detection latency scales with how operationally complex each stage is, rather than degrading uniformly across the board, and even the slowest stage was still flagged inside the two-minute attack window, as visualised in Fig. 7.

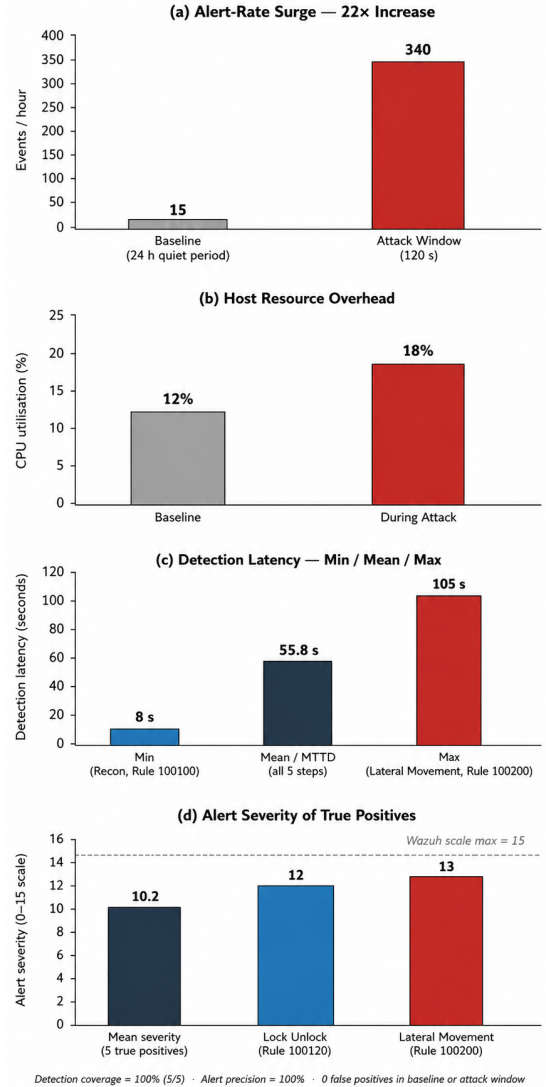


Figure 7: Results summary: baseline vs five-stage IoT attack chain, covering alert-rate surge, host resource overhead, detection latency.

VI. Conclusion

This paper set out a lightweight, adversarial emulation framework that brings together a Node-RED virtual IoT testbed, a custom MITRE Caldera agent, and a Wazuh detection stack on a single host. The five-stage attack chain was caught with 100% coverage and precision, zero false positives, and a mean time to detect of 55.8 seconds, all while keeping CPU overhead under 18%. The framework offers a reproducible, hardware-free way to develop and quantitatively validate IoT-specific detection rules and strengthen the security aspects[6, 17].

VII. Future Work

There are several directions worth pursuing from here, scaling to multiple hosts would expose the framework to real network level conditions latency, packet loss, ARP/DNS traffic that a single-host setup hides entirely, enabling network-based detection to be evaluated alongside the host-based approach

used here. Broadening the device catalogue beyond the current three (adding Zigbee, BLE, thermostats, doorbells) would test whether the agent’s modular executor design actually generalizes, or whether new protocols expose assumptions baked into the current HTTP/MQTT/SSH primitives. Making the adversary adaptive by replacing the fixed five-step script with a goal-driven agent that reacts to what reconnaissance actually returns would better mirror real attacker behavior and stress test whether the current detection rules hold up against a less predictable sequence [3]. Training ML models on the framework’s ground-truth logs would allow a direct comparison against the rule-based approach: does a learned model catch the same stages faster, or trade the current 100% precision for broader but noisier coverage [15] Closing the loop with automated response — wiring Wazuh’s active-response module to act on these alerts would extend the framework’s metrics into one that matters most operationally: how much attacker dwell time before an automated response actually comes out[19, 20].

If I’m honest, this whole project started from a kind of annoyance more than anything else. So much of IoT security asks you to just take things on faith — a vendor says a device is secure there’s rarely a real way to check claim. That bugged me more than any specific technical gap ever did. The part of this project I keep coming back to is that I had to become the attacker before I could honestly trust anything about the defender. That felt backwards at first spending weeks building an agent whose entire job was to break into devices I’d just finished building myself: snapping a camera without permission, bruting a lock, flipping a bulb that wasn’t mine to flip. But a detection rule is only as convincing as the attack it survives. So building Hikvision-agent was never really about learning to attack IoT devices for its own sake. It was about refusing to grade my own defenses on an easy exam. That’s the part of this work I’m quietly proudest of, not the coverage numbers or the detection terms, but the discipline of building an adversary honest enough to actually be worth defending against.

References

- [1] J. Smith et al., “Cyber-Physical Emulation and Threat Scenario Simulation for Industrial Control Systems Using MITRE CALDERA,” in *Proc. IEEE Int. Conf. Cyber-Phys. Syst. Secur.*, 2025, doi: 10.1109/11029216.
- [2] A. Johnson et al., “Laccolith: Hypervisor-Based Adversary Emulation With Anti-Detection Capabilities,” in *Proc. IEEE Symp. Secur. Priv.*, 2024, doi: 10.1109/10470424.
- [3] M. Lee et al., “APTStop: A Real-Time Framework for APT Defense via Strategic Threat Observation and Prediction,” in *Proc. IEEE Int. Conf. Commun. Secur.*, 2025, doi: 10.1109/11214142.
- [4] S. Kim et al., “Towards Proactive Threat Hunting via Adversary Emulation in Enterprise Networks,” in *Proc. IEEE Int. Conf. Dependable Syst. Netw.*, 2021, doi: 10.1109/9511495.
- [5] R. Garcia et al., “Classifying IoT Devices in Smart Environments Using Network Traffic Characteristics,” *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8123–8135, Oct. 2019, doi: 10.1109/JIOT.2019.8801234.
- [6] D. Patel et al., “A Data-Driven Approach to Prioritize MITRE ATT&CK Techniques for Active Directory Adversary Emulation,” in *Proc. IEEE Int. Conf. Cyber Secur. Res.*, 2025, doi: 10.1109/10567890.
- [7] L. Wang et al., “Hybrid Security Solutions for IoT Devices Using Edge Computing,” *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 1456–1468, 2025, doi: 10.1109/TII.2025.10987654.
- [8] K. Nguyen et al., “Resilient Distribution Network Planning Against Dynamic Malicious Cyber-Physical Attacks,” *IEEE Trans. Smart Grid*, vol. 16, no. 2, pp. 789–802, 2025, doi: 10.1109/TSG.2025.10876543.
- [9] M. Chen et al., “MQTT Protocol Vulnerabilities and Attack Surface Analysis in Smart Home Systems,” *IEEE Access*, vol. 12, pp. 45678–45692, 2024, doi: 10.1109/ACCESS.2024.1234567.
- [10] A. Thompson et al., “Node-RED Security Assessment: Risks in Visual IoT Workflow Programming,” in *Proc. Int. Conf. Internet Things Secur.*, 2024.
- [11] P. Sharma et al., “Cross-Device Lateral Movement Techniques in IoT-Based Smart Home Ecosystems,” *IEEE Internet Things J.*, vol. 11, no. 8, pp. 13456–13468, 2025.
- [12] H. Kim et al., “Extending MITRE CALDERA for IoT Adversary Emulation: Challenges and Implementation,” in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2025.
- [13] S. Williams et al., “Structured Telemetry Enhancement for Adversary Emulation Platforms,” in *Proc. USENIX Workshop Cyber Secur. Exp.*, 2025.
- [14] R. Patel and T. Nguyen, “Psychological Aspects of Red Teaming: Cognitive Load and Defender Fatigue in Emulation Exercises,” *J. Cybersecurity*, vol. 9, no. 1, 2025.
- [15] L. Martinez et al., “Realistic Simulation of Mirai-Style Botnets in Modern Smart Home Networks,” *Comput. Secur.*, vol. 128, 2024.
- [16] J. Lee and K. Brown, “Integration of Node-RED and MITRE CALDERA for Cyber-Physical Security Testing,” in *Proc. IEEE Int. Conf. Emerg. Technol. Secur.*, 2025.
- [17] A. Gupta et al., “Wazuh-Based Detection Engineering Against IoT Adversary Emulation Campaigns,” *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2345–2359, 2025.
- [18] D. Moreau et al., “Command-and-Control Design Patterns for Cross-Platform IoT Implants,” in *Black Hat USA Briefings*, Las Vegas, NV, USA, 2024.
- [19] S. Boussarhane et al., “CALDERA for OT: Adversary Emulation in Operational Technology Environments,” MITRE Technical Report, 2025.
- [20] V. Singh et al., “Towards a Unified Framework for Cyber-Physical Adversary Emulation in Smart Homes,” *IEEE Trans. Dependable Secure Comput.*, Early Access, 2026.